

Watchdog Python Full Notes (Basic to Advanced)

1. Introduction

Watchdog is a Python library used to monitor file system events such as file creation, modification, deletion, and movement in real time.

2. Installation

```
pip install watchdog
```

3. Core Concepts

Observer monitors the file system. Event Handler processes detected events. Scheduler links observer and handler.

4. Event Types

on_created, on_deleted, on_modified, on_moved

5. Basic Example

```
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import time

class Handler(FileSystemEventHandler):
    def on_created(self, event):
        print("Created:", event.src_path)

observer = Observer()
observer.schedule(Handler(), path='.', recursive=True)
observer.start()

while True:
    time.sleep(1)
```

6. Filtering Files

```
def on_modified(self, event):
    if event.src_path.endswith(".py"):
        print("Python file changed")
```

7. Advanced: Multiple Handlers

```
observer.schedule(Handler1(), path='folder1')
observer.schedule(Handler2(), path='folder2')
```

8. Threading Usage

Heavy tasks should be handled with threading to avoid blocking the observer loop.

9. Logging System

```
import logging
logging.basicConfig(level=logging.INFO)
```

10. Real Projects

Auto backup system, Django auto reload, log monitor, security watcher.

11. Performance Optimization

Use non-recursive monitoring when possible. Add sleep delay. Filter unnecessary events.

12. OS Support

Windows: ReadDirectoryChangesW, Linux: inotify, macOS: FSEvents

13. Common Errors

Multiple triggers, high CPU usage, missing permissions.

14. Comparison with Polling

Watchdog is real-time and efficient, polling is slower and resource-heavy.

15. Conclusion

Watchdog is powerful for automation and real-time systems.